A EXEMPLAR QUERIES

```
-- Extract keywords from a content field
SELECT id, s'Extract the main keywords from {content}' AS
↪  extracted_keywords
FROM documents;
-- Create a semantic summary from multiple fields
SELECT s'A summary of the event based on {title} and
↪  {description}'
AS event_summary FROM news_articles;
```

Figure 9: Example of Semantic Projection

```
-- Filter rows based on the sentiment of a specific field
SELECT * FROM product_reviews
WHERE s'The {review_text} expresses positive sentiment';
```

Figure 10: Example of Semantic Filter

```
-- Analyze the language style of the conference paper title
WITH title_style AS (
    SELECT Title, s'Analyze the language style features of paper
    ↪  title {Title}.
    Describe the key characteristics such as: formality level,
    technical terminology usage, length, structure, and word
    ↪  choice.'
    AS style_features
    FROM Paper WHERE ConferenceId != 0 AND Title IS NOT NULL
)
SELECT sem_agg(s'Based on these conference paper titles {Title}
and their style analyses {style_features}, summarize the overall
language style and terminology characteristics of conference
↪  paper titles.')
FROM title_style;
```

Figure 11: Example of Semantic Aggregate

```
-- Join two tables based on semantic similarity of their text
↪  fields
SELECT * FROM news_articles a JOIN blog_posts b
ON s'the topic of {a.headline} is related to {b.title}';
-- A complex hybrid join with both syntactic and semantic
↪  conditions
SELECT e.employee_id, eh.manager_id
FROM employees e INNER JOIN employee_hierarchy eh
ON e.manager_id = eh.employee_id
AND s'the {e.title} is a technical role and the hierarchy level
↪  {eh.level} < 3';
```

Figure 12: Example of Semantic Join

B ALGORITHMS

Algorithm 4 generates a set of candidate execution paths with dynamic pruning guided by predicate correlations. Given a set of predicates Σ, the algorithm first sorts all predicates in ascending order of selectivity to construct a baseline sequential execution path,

Algorithm 2: Natural Language Expression Preprocessing and Reduction Process

Input : userInputNLE, dbSchema
Output : finalSQLFragments or processedNLE

// Step 1: Preprocessing with Database Context
intermediateNLE ← CleanRedundancies(userInputNLE)
cleanedNLE ← CorrectFields(intermediateNLE, dbSchema)
// Step 2: Reduction Assessment
prelimStatus ← AssessReduction(cleanedNLE)
if *prelimStatus = "not_reducible"* **then**
  return processedNLE ← cleanedNLE
reductionStatus ← AssessDbReduction(cleanedNLE, dbSchema)
if *reductionStatus = "not_reducible"* **then**
  return processedNLE ← cleanedNLE
candidateSQL ← GenerateSQL(cleanedNLE, dbSchema)
expandedSQL ← ExpandValues(candidateSQL, dbSchema)
return finalSQLFragments ← expandedSQL

Algorithm 3: Expression Exploration and Pairwise Similarity Estimation

Input : $\Sigma = \{\sigma_1, \ldots, \sigma_n\}$, sample chunk $\mathcal{D}_1$
Output : Selectivities s, result vectors R, similarity matrix MCC

for $i \leftarrow 1$ *to* n **do**
  $R[i] \leftarrow \text{ExecutePredicate}(\sigma_i, \mathcal{D}_1)$;
  $S[i] \leftarrow \frac{\text{CountTrue}(R[i])}{|\mathcal{D}_1|}$;
for $i \leftarrow 1$ *to* n **do**
  for $j \leftarrow i+1$ *to* n **do**
    $(TP, TN, FP, FN) \leftarrow \text{ConfusionCounts}(R[i], R[j])$;
    $\text{MCC}[i][j] \leftarrow \text{Matthews}(TP, TN, FP, FN)$;
return S, R, MCC

Algorithm 4: Candidate Path Generation with Dynamic Pruning

Input : s, MCC, Σ, μ_{thresh}, B, V
Output : Candidate paths $\mathcal{P}$

$p_{\text{sorted}} \leftarrow \text{SortAscendingBySelectivity}(\Sigma, S)$;
$\mathcal{P} \leftarrow \{\text{SeqPath}(p_{\text{sorted}})\}$;
$\mathcal{F} \leftarrow \emptyset$;
for $i \leftarrow 1$ *to* n **do**
  for $j \leftarrow i+1$ *to* n **do**
    if $\text{MCC}[i][j] > \mu_{thresh}$ **then**
      $\mathcal{F} \leftarrow \mathcal{F} \cup \{(\sigma_i, \sigma_j)\}$;
for *each* $(\sigma_a, \sigma_b) \in \mathcal{F}$ **do**
  $\sigma_a \oplus \sigma_b \leftarrow \text{Fuse}(\sigma_a, \sigma_b)$;
  $\mathcal{P} \leftarrow \mathcal{P} \cup \{\text{SeqPath}(\text{ReplaceOnce}(p_{\text{sorted}}, \{\sigma_a, \sigma_b\} \rightarrow \sigma_a \oplus \sigma_b))\}$;
return $\mathcal{P}$

which is always included in the candidate set $\mathcal{P}$. It then examines pairwise predicate correlations using a precomputed Matthews Correlation Coefficient (MCC) matrix and retains only those predicate pairs whose correlation exceeds a threshold μ_{thresh}, effectively pruning weakly related combinations. For each retained pair (σ_i, σ_j), the algorithm applies a fusion function to form a fused predicate $\sigma_i \oplus \sigma_j$ and generates a new candidate path by replacing exactly

Algorithm 5: Path Exploration and Best-Path Selection

Input : Paths $\mathcal{P}$, sample chunk $\mathcal{D}_2$, threshold τ_{acc}
Output : Best path $p^\star$

Execute reference path to get ground truth G ;
for *each* $p \in \mathcal{P}$ **do**
  $(\text{lat}, \text{cost}, \text{sel}) \leftarrow \textsc{Run}(p, \mathcal{D}_2)$;
  $\text{acc} \leftarrow \textsc{Accuracy}(\text{sel}, G)$;
$\mathcal{F} \leftarrow \textsc{ParetoFrontier}(\mathcal{P}, \text{lat}, \text{cost})$;
$p^\star \leftarrow \arg\min_{p \in \mathcal{F}} \mathcal{M}[p].\text{lat}$;
return $p^\star$

one occurrence of the original predicates in the sorted sequence with the fused predicate, while preserving the relative order of all remaining predicates. All generated paths are collected into $\mathcal{P}$ and treated as candidate paths for subsequent path exploration phase.

Algorithm 5 explores the candidate execution paths and selects the best path for the exploitation phase. Given a set of candidate paths $\mathcal{P}$ and a sampled data chunk $\mathcal{D}_2$, the algorithm first executes a reference path to obtain ground-truth results. Each candidate path is then executed on $\mathcal{D}_2$ to measure its latency, execution cost, and selection output, from which accuracy is computed against the ground truth. Based on the collected latency and cost metrics, the algorithm constructs a Pareto frontier to retain only non-dominated paths. Among these Pareto-optimal candidates, the path with the minimum latency or minimum cost is selected as the final best path $p^\star$ and returned for execution.

C ADDITIONAL EXPERIMENTAL RESULTS

C.1 Impact of Execution Optimization

To assess the utility of execution optimization techniques, we compare the performance of Sema under four configurations: (1) *w/o optimization*, (2) *w/ prompt batching* with a batch size of 16, (3) *w/ operator fusion*, and (4) *w/ both batch and fusion*. All experiments are conducted using two LLM backbones, Gemma-3 12B and GPT-4.1, where GPT-4.1 is invoked remote API and Gemma-3 12B is invoked both remote API and locally.The outputs from GPT-4.1 serve as the ground truth for evaluation. We use four queries, Q8, Q13, Q14 and Q16 with distinct combinations of operators to evaluate the above configurations.

Experimental Results. Table 8 and Table 7 summarize the results for the four queries under the four system configurations. Both batching and fusion substantially improve execution efficiency over the no-optimization baseline, though their effects on cost, latency, and result quality vary with operator composition.

When moving from API-based execution to local deployment, we observe that the qualitative impact of batching and fusion on cost and result quality remains largely consistent. However, their effect on latency differs substantially. In API-based settings, latency is dominated by external service constraints such as rate limits, where excessive invocation frequency significantly reduces throughput. In contrast, local execution eliminates such rate-limiting effects, shifting the primary latency determinants to a combination of LLM invocation count and per-invocation execution time.

Under local execution, fusion consistently reduces latency by lowering the number of LLM calls, despite a modest increase in per-call input tokens due to more complex fused instructions. Except for cases with strict output format requirements—such as Q8 with consecutive filters—fusion generally yields net latency improvements. Batching, however, exhibits a different behavior: although it reduces the total number of LLM invocations, the substantially increased input and output token lengths per call lead to longer per-invocation execution times. As a result, overall latency under batching may slightly increase in local execution, despite fewer calls.

Under the API-based execution setting,batching notably reduces the number of LLM invocations, leading to substantial cost and latency improvements across all queries. For example, in Q8, total execution time drops by 17× for Gemma-3 12B and 12× for GPT-4.1, with a 40–50% cost reduction, which are primarily by reducing LLM invocations. However, batching often comes at a quality trade-off, as reflected by lower accuracy and F1 scores, especially in *SemProj* first pipelines (e.g., Q13, Q14), where the outputs of *SemProj* become less informative due to its condensed prompt in the overall batch. Fusion is generally more effective at preserving result quality while offering moderate improvements in cost and latency. For instance, in Q13, fusion cuts cost by over 60% and improves both accuracy and F1. This benefit is most pronounced in *SemProj* first pipelines, where fusion allows downstream operators to access richer, uncompressed input. In *SemFilter* first pipelines like Q8, the impact on cost is smaller, but result quality remains stable.

When batching and fusion are combined, their benefits do not simply add up: while cost and latency are further reduced, overall quality tends to decrease, as observed in Q8 and Q13. This suggests that aggressive batching can undermine the contextual gains achieved by fusion. Across all configurations, GPT-4.1 consistently outperforms Gemma-3 12B in both accuracy and F1, demonstrating greater robustness under fusion and batching. These results suggest that larger models are more resilient to prompt compression and inter-operator interference, maintaining stable quality even as the number of invocations is reduced.

Table 7: Impact of Prompt Batching & Operator Fusion (Local)

Model Type	Technique	IT (K)	OT (K)	Cost($)	Lat. (s)	Acc	F1
Q8 (Consecutive *SemFilter*)							
Gemma3-12b	Raw	2019	27	0.0843	58.35	0.9324	0.8015
	Batch16	1013	100	0.0535	99.60	0.8374	0.6626
	Fusion	2046	286	0.1190	165.48	0.8876	0.7520
	Batch+Fusion	355	301	0.0533	144.98	0.8805	0.7169
Q13 (*SemProj* ⊕ *SemFilter*)							
Gemma3-12b	Raw	113	14	0.0063	12.79	0.6751	0.7357
	Batch16	52	11	0.0036	14.12	0.5329	0.5517
	Fusion	55	0.6	0.0022	4.65	0.7904	0.8448
	Batch+Fusion	34	2	0.0017	3.92	0.6572	0.8087
Q14 (*SemProj* ⊕ *SemProj*)							
Gemma3-12b	Raw	216	39	0.0137	27.07	0.9160	0.7123
	Batch16	102	33	0.0084	24.48	0.8932	0.8304
	Fusion	151	7	0.0070	19.02	0.9279	0.7377
	Batch+Fusion	64	9	0.0037	9.60	0.9060	0.6793
Q16 (*SemFilter* ⊕ *SemProj*)							
Gemma3-12b	Raw	1329	23	0.0562	9.87	0.9938	0.9878
	Batch16	712	45	0.0344	11.96	0.8899	0.8028
	Fusion	1316	21	0.0553	8.13	0.9951	0.9904
	Batch+Fusion	473	34	0.0233	10.69	0.8956	0.8234

Table 8: Impact of Prompt Batching & Operator Fusion (Input Tokens (IT), Output Tokens (OT), total latency (Lat. in seconds), The first and second best results of cost and total latency are highlighted and underlined, respectively)

Model Type	Technique	IT (K)	OT (K)	Cost($)	Lat. (s)	Acc	F1
Q8 (Consecutive *SemFilter*)							
Gemma3-12b	Raw	2009	41	0.0857	4903.96	0.9245	0.8020
	Batch16	1011	100	0.0535	282.38	0.8592	0.6552
	Fusion	2046	282	0.1185	1932.52	0.8774	0.7206
	Batch+Fusion	355	301	0.0533	508.05	0.8286	0.7125
GPT-4.1	Raw	1994	26	4.1996	3833.65	1.0000	1.0000
	Batch16	1021	37	2.3419	329.17	0.8635	0.7067
	Fusion	2046	258	6.1558	1884.87	0.9087	0.7934
	Batch+Fusion	355	293	3.0521	307.13	0.8931	0.7522
Q13 (*SemProj* ⊕ *SemFilter*)							
Gemma3-12b	Raw	113	14	0.0063	126.67	0.9218	0.7332
	Batch16	53	12	0.0037	32.84	0.5180	0.5452
	Fusion	54	0.8	0.0023	57.56	0.7934	0.8473
	Batch+Fusion	33	2	0.0017	10.19	0.6647	0.7787
GPT-4.1	Raw	125	25	0.4531	191.10	1.0000	1.0000
	Batch16	59	15	0.2454	45.60	0.7695	0.8421
	Fusion	54	0.6	0.1150	75.00	0.8308	0.8889
	Batch+Fusion	33	0.8	0.0744	6.81	0.7246	0.8274
Q14 (*SemProj* ⊕ *SemProj*)							
Gemma3-12b	Raw	205	34	0.0126	139.41	0.9121	0.8015
	Batch16	97	27	0.0075	24.76	0.8923	0.7094
	Fusion	151	2.5	0.0063	63.58	0.9249	0.8242
	Batch+Fusion	64	6	0.0034	8.89	0.9078	0.7472
GPT-4.1	Raw	214	42	0.7711	182.60	1.0000	1.0000
	Batch16	60	25	0.3260	38.79	0.9284	0.8079
	Fusion	151	3.9	0.3337	75.87	0.9682	0.9241
	Batch+Fusion	64	5.6	0.1730	14.06	0.9266	0.8034
Q16 (*SemFilter* ⊕ *SemProj*)							
Gemma3-12b	Raw	1216	19	0.0513	1387.38	0.8584	0.6287
	Batch16	707	42	0.0338	88.34	0.8945	0.8087
	Fusion	1281	30	0.0553	720.00	0.9938	0.9878
	Batch+Fusion	473	33	0.0234	93.95	0.9103	0.8590
GPT-4.1	Raw	1349	23	2.8905	1598.62	1.0000	1.0000
	Batch16	675	28	1.5759	156.53	0.9430	0.8839
	Fusion	1316	36	2.9212	1373.96	0.9961	0.9923
	Batch+Fusion	472	29	1.1792	97.49	0.9940	0.9887

D DATASET & BENCHMARK QUERIES

Table 10: Profile of Query Set

Category	Subcategory	Query ID
Single Filter/Proj.	Single Operators	Q1–Q5
Consecutive Filters	Filter → Filter	Q6–Q10
Proj.-First	Proj. → Filter	Q11–Q13
	Proj. → Proj.	Q14–Q15
Filter-First	Filter → Proj.	Q16–Q18
	Filter → Aggregate	Q19–Q20

```
SELECT id FROM (
   SELECT TweetID AS id, sentiment, text FROM twitter LIMIT 32768
) WHERE s'The sentiment label rules are:
- sentiment > 0 -> Positive
- sentiment = 0 -> Neutral
- sentiment < 0 -> Negative
Task: Judge whether the {sentiment} label is consistent with the
↪  {text} content.');
```

Figure 13: Q1: Sentiment analysis

```
SELECT s'Classify the given {"Product Name"} into exactly one of
↪  these categories:
[Furniture, Office Supplies, Technology].
Output only the category name, with no explanations, no quotes,
↪  and no extra words.
Example:
Input: Hon Metal Bookcases, Black Output: Furniture
Input: AT&T CL2909 Output: Technology
Input: Belkin F9S820V06 8 Outlet Surge Output: Office Supplies'
↪  AS predicted_category FROM product;
```

Figure 14: Q2: Classify product into one of three categories

```
SELECT iucr_no FROM IUCR
WHERE s'The index_code label rules are:
- index_code = I -> Indexed (severe, such as murder, rape, arson,
↪  and robbery)
- index_code = N -> Non-Indexed (less serve, such as vandalism,
↪  weapons violations, and peace disturbance)
Task: Determine whether the crime''s {primary_description} and
↪  {secondary_description} indicates this crime is consisted
↪  with {index_code}');
```

Figure 15: Q3: Determine whether crime's description is consistent with index code

```
SELECT s'Task: Classify the given violation {description} into
↪  exactly one of these categories:
- Low Risk
- Moderate Risk
- High Risk
Output only the risk category, with no explanations, no quotes,
↪  and no extra words.
Example:
Input: Inadequate and inaccessible handwashing facilities Output:
↪  Moderate Risk
Input: Unclean or degraded floors walls or ceilings Output: Low
↪  Risk
Input: Unclean or unsanitary food contact surfaces Output: High
↪  Risk' AS predicated_category, risk_category AS ground_truth
↪  FROM violations;
```

Figure 16: Q4: Determine the risk degree according to violation description

Table 9: Datasets and Queries Used in the Experiments

Dataset	# Tables	Table (# Rows)	Attributes	Queries
Appstore	2	user_review (10,840), playstore (64,286)	user_review.Translated_Review	Q7, Q8
Chicago_Crime	1	IUCR (401)	primary_description, secondary_description, index_code	Q3
Superstore	1	product (5,298)	productName, category	Q2
Food_Inspection	1	violations (36,050)	description, risk_category	Q4
Food_Inspection_2	1	violation (525,709)	inspector_comment, fine	Q5
Social_Media	3	twitter (99,901), user (99,260), location (6,211)	twitter.text	Q1, Q9, Q16–Q18
Movies_4	7	movie (4,627), country (88), production_country (6,436), genre (20), movie_genres (12,160), language (88), movie_languages (11,740)	movie.overview	Q10, Q13–Q15
CoinMarketCap	2	coins (8,927), historical (4,441,972)	coins.description	Q6, Q11, Q12
Authors	5	Paper (2,254,920), Journal (15,151), Conference (4,545), Author (247,030), PaperAuthor (2,315,574)	Paper.Keyword, Paper.Title	Q19, Q20

```
SELECT inspection_id FROM (
    SELECT distinct inspection_id,inspector_comment, fine FROM
    ↪  violation
    WHERE inspector_comment IS NOT NULL
      AND fine IN [100, 250, 500]
    LIMIT 16384
)
WHERE s'The fine of more serious food safety problems will be
↪  higher.
The fine label rules are:
- fine = 100 -> Minor
- fine = 250 -> Serious
- fine = 500 -> Critical
Fine example:
1. {inspector_comment}: REPAIR THE DRAIN STOPPER FOR THE RIGHT
↪  COMPARTMENT OF THE 3 COMPARTMENT SINK SO THAT IT CAN BE USED
↪  TO HOLD WATER. {fine}: 100
2. {inspector_comment}: NO CHEMICAL TEST STRIPS ON PREMISES-MUST
↪  PROVIDE. SERIOUS VIOLATION 7-38-030. {fine}: 250
3. {inspector_comment}: FOUND WALK IN COOLER TEMPERATURE AT 45.3F.
↪  CRITICAL VIOLATION.7-38-005( A)  {fine}: 500
Task: Determine the {inspector_comment} is consistent with the
↪  {fine}. Output true or false only');
```

Figure 17: Q5: Determine the risk degree according to violation description

```
SELECT coins.id AS coin_id FROM coins
WHERE category = 'coin' AND description is NOT NULL
AND s'The {description} shows that this cryptocurrency has a big
↪  supply amount.'
AND s'The {description} indicates that this cryptocurrency is
↪  still in circulation.'
AND s'It can be inferred from the {description} that the price of
↪  this cryptocurrency is rising.'
```

Figure 18: Q6: Find cryptocurrencies that are still in circulation and have a big supply amount with a rising price

```
SELECT ur.id AS id
FROM user_reviews ur
JOIN playstore p ON ur.App = p.App
WHERE p.Category = 'TOOLS'
AND s'{ur.Translated_Review} is a complete sentence and has
↪  informative content'
AND s'{ur.Translated_Review} is a positive user review'
AND s'{ur.Translated_Review} shows that this app is a useful
↪  tool';
```

Figure 19: Q7: Find positive, complete and informative user reviews of TOOLS apps that indicate the app is a useful tool

```
SELECT ur.id AS id
FROM user_reviews ur
JOIN playstore p ON ur.App = p.App
WHERE p.Category = 'GAME'
AND s'{Translated_review} is a complete sentence and has
↪  informative content'
AND s'{Translated_review} is a positive user review'
AND s'{Translated_Review} shows that users have a good experience
↪  and fun with this app';
```

Figure 20: Q8: Find positive, complete and informative user reviews of GAME apps that indicate users have a good experience and fun with the app

```
SELECT ur.id AS id
FROM user_reviews ur
JOIN playstore p ON ur.App = p.App
WHERE p.Category = 'FAMILY'
AND s'{Translated_Review} is a complete sentence and has
↪  informative content'
AND s'{Translated_Review} is a positive user review'
AND s'{Translated_Review} shows that this app is suitable for
↪  kids';
```

Figure 21: Q9: Find positive, complete and informative user reviews of FAMILY apps that indicate the app is suitable for kids

```sql
SELECT m.movie_id
FROM movie m
WHERE s'The {m.overview} suggests that the movie is an action
↪  movie'
AND s'The {m.overview} shows that the movie focuses on a central
↪  role'
AND s'The {m.overview} shows that the movie is based on
↪  historical events';
```

Figure 22: Q10: Find action movies that focuse on a central role and are based on historical events

```sql
SELECT coin_id FROM
(SELECT s'According to {c.description},introduce the situation of
↪  cryptocurrency circulation' AS circulation, c.id AS coin_id
    FROM coins c JOIN historical h ON h.coin_id = c.id
    AND h.date = '2019-01-09'
    AND c.status IN ('untracked','inactive')
    AND c.description IS NOT NULL)
WHERE s'{circulation} indicates cryptocurrency is under
↪  development or in testnet.';
```

Figure 23: Q11: Find cryptocurrencies that were inactive or untracked on Jan 9, 2019, had missing price data, and whose description suggests they were still under development or operating on a testnet

```sql
SELECT coin_id FROM
(SELECT s'From the description {description}, extract information
↪  about the underlying technology of cryptocurrency, focusing
↪  on whether it mentions a proprietary blockchain, a native
↪  consensus mechanism, or an independent network. Provide a
↪  concise summary of these technological characteristics.' AS
↪  tech_features,
    coins.id AS coin_id
    FROM coins c JOIN historical h ON h.coin_id = c.id
    WHERE c.platform_id IS NOT NULL AND c.description IS NOT NULL
    AND h.circulating_supply IS NOT NULL
    AND h.total_supply IS NOT NULL
    AND h.date = '2019-01-10'
    )
WHERE s'Based on the technological characteristics
↪  {tech_features}, determine whether cryptocurrency has its own
↪  independent blockchain or consensus mechanism';
```

Figure 24: Q12: Find coins whose summarized technological features indicate the cryptocurrency has its own blockchain or consensus mechanism

```sql
SELECT movie_id FROM
(SELECT s'Summarize the geographic and temporal background of the
↪  movie according to {overview}.' AS background, m.movie_id AS
↪  movie_id
    FROM movie m
    SEMI JOIN (
      SELECT mg.movie_id
      FROM movie_genres mg JOIN genre g ON mg.genre_id =
      ↪  g.genre_id
      WHERE g.genre_name IN ('Thriller','Comedy','Western')
    ) mg ON m.movie_id = mg.movie_id
    WHERE revenue > budget
      AND length(overview) < 255
 )
WHERE s'{background} indicates the story is set in modern times
↪  rather than historical periods.';
```

Figure 25: Q13: Find profitable movies whose background indicates the story is set in modern times rather than historical periods with limited genres

```sql
SELECT movie_id, s'Classify {plot} into one of the following
↪  categories:
[Sci-Fi, Crime, Thriller, Romance, Drama, Horror, Comedy, War,
↪  Fantasy, Family, Animation, Biography, Other]
Output only the category that best fits the movie, without any
↪  other text or reasoning content.' AS category FROM
(SELECT s'Summarize the main plot of {overview}' AS plot, movie_id
    FROM movie m
    SEMI JOIN (
        SELECT ml.movie_id
        FROM movie_languages ml
        JOIN language l ON ml.language_id = l.language_id AND
        ↪  l.language_name = 'English'
    ) ml ON m.movie_id = ml.movie_id AND movie_status =
    ↪  'Released' AND popularity > 10 AND length(overview) < 255
    ↪  AND length(overview) > 0
);
```

Figure 26: Q14: Summarize the main plot of English released popular movies and classify plots into genres

```
SELECT movie_id,
s'This is a multi-label classification task. Given the {conflict},
↪  identify what the main character is in conflict with, and
↪  classify the it in the format "character vs X", where X is
↪  one or more of: [Character, Self, Society, Nature, Technology,
↪  Machine, Supernatural, Fate, Other].
Output only X, as a comma-separated list (e.g., Self, Society).
Do not include explanations or reasoning.' AS categories
FROM (SELECT movie_id, s'Extract all conflicts from movie
↪  {overview} as one sentence.' AS conflict
    FROM movie
    SEMI JOIN
        (SELECT pc.movie_id
         FROM production_country pc JOIN country c
         ON pc.country_id = c.country_id AND c.country_iso_code =
         ↪  'US')
    pc ON movie.movie_id = pc.movie_id AND (movie.release_date
    ↪  BETWEEN '2000-01-01' AND '2015-12-31') AND
    ↪  movie.vote_average > 6.0 AND length(overview) < 255;
```

Figure 27: Q15: Extract all conflicts from high-rated movies between 2000 and 2015 and classify conflicts into categories

```
SELECT s'From the job posting tweet {t.text}, extract the
↪  specific job titles and required skills mentioned. Return
↪  them as a comma-separated list (e.g., Software Engineer,
↪  Python, AWS, Machine Learning) Focus on concrete job titles
↪  and technical skills' AS info,t.TweetID AS twitter_id
FROM twitter t JOIN user u
On t.UserID = u.UserID
WHERE user.Gender = 'Female'
AND t.Sentiment = 0.0
AND t.TweetID IS NOT NULL
AND Klout > 50
AND s'Tweets with content as {text} are about job postings,
↪  recruitment announcements, hiring notices, or career
↪  opportunities related to AWS or cloud computing (has contents
↪  like hiring, job opening, career opportunity, we are looking
↪  for, join our team, etc.)';
```

Figure 28: Q16: Filter neutral tweets which are posted by females and related to recruitment and extract concrete job titles and technical skills.

```
SELECT s'Based on the tweet content :"{text}", identify the main
↪  type of security or privacy concern being expressed. Classify
↪  text into categories like data breach, unauthorized access,
↪  encryption weakness, compliance violation, identity theft,
service vulnerability, configuration error, monitoring concerns,
↪  or other specific security/privacy concern types.' AS type,
twitter.TweetID AS twitter_id
FROM twitter
WHERE Sentiment < -2.5 AND Klout > 20
AND t.TweetID IS NOT NULL
AND s'Tweet "{text}" discusses security concerns, privacy issues,
↪  data protection, cybersecurity threats, authentication
↪  problems, or confidentiality matters related to AWS
↪  services';
```

Figure 29: Q17:Find negative tweets with high influence that are related to potential risks of AWS services and identify the main type of security or privacy concern

```
SELECT s'From this tweet "{t.text}", extract the main topic or
↪  subject being discussed about programming/development. Return
↪  a concise topic label like performance optimization, learning
↪  resources, job opportunities, technical issues, new features,
↪  etc.'
twitter.TweetID AS twitter_id
FROM twitter t JOIN location l
On t.LocationID = l.LocationID
WHERE l.Country IN ('United States', 'Canada', 'United Kingdom')
AND Sentiment >= 3.0 AND Reach>1000
AND t.TweetID IS NOT NULL
s'Filter tweets "{t.text}" that mention amazon web services or
↪  cloud services, including specific AWS products or general
↪  cloud computing issues';
```

Figure 30: Q18: Filter tweets related to AWS products and extract discussed topics or subjects

```
SELECT sem_agg('Summarize the main research theme reflected by
↪  the following keywords: {p.Keyword}. Respond with a concise,
↪  high-level scientific topic',Keyword) FROM Paper p
JOIN Journal j
ON j.Id= P.JournalId
WHERE p.Year BetWEEN 2000 AND 2011
AND j.ShortName IN ('TODS','VLDB','TKDE','JDM','DKE')
AND s'Based on the keywords "{p.Keyword}" , determine if this
↪  paper is related to database systems, data management, or
↪  query processing';
```

Figure 31: Q19: Find papers of Database journals and related to database systems, data management, or query processing and aggregate these papers' keywords to find main research themes

```sql
SELECT ConferenceId, sem_agg('Identify and summarize the major
↪  interdisciplinary research trends from the following keywords:
↪  {p.Keyword}. Return only one sentence.',Keyword) AS trends
FROM paper p
JOIN Conference c ON p.ConferenceId = c.Id
SEMI JOIN (
SELECT DISTINCT pa.PaperId
  FROM PaperAuthor pa
  WHERE pa.AuthorId IS NOT NULL
    AND (LOWER(COALESCE(pa.Affiliation,'')) LIKE '%institute%'
    OR LOWER(pa.Affiliation) LIKE '%laboratory%'
    OR LOWER(pa.Affiliation) LIKE '%research center%')
) filtered_papers
ON p.Id = filtered_papers.PaperId
AND p.Keyword IS NOT NULL
AND length(p.Keyword) > 0
WHERE s'Determine whether the paper with keyword as {Keyword} is
↪  related to interdisciplinary methods.'
GROUP BY ConferenceId
HAVING COUNT(*) >= 3 AND trends IS NOT NULL
```

Figure 32: Q20: Find papers whose authors are in institute, laboratory or research center and identify interdisciplinary research trends

```sql
-- Note: Introduces redundant expressions - repeated field
↪  references ({primary_description}, {secondary_description},
↪  {index_code} each appear twice), redundant modifiers ("after
↪  reviewing...again for consistency"), and complex sentence
↪  structures.
SELECT COUNT(*) FROM chicago_crime.IUCR
WHERE s'The index_code label rules are:\n\
- index_code = I -> Indexed (severe, such as murder, rape, arson,
↪  and robbery)\n\
- index_code = N -> Non-Indexed (less serve, such as vandalism,
↪  weapons violations, and peace disturbance)\n\
Based on these rules, the crime with primary description
↪  {primary_description} and secondary description
↪  {secondary_description} should be checked against index code
↪  {index_code}, and after reviewing the primary description
↪  {primary_description} again for consistency, it should be
↪  determined whether this crime is consistent with the assigned
↪  index code {index_code} according to the established
↪  classification criteria.';
```

Figure 33: Q3 before Expression Reduction

```sql
-- Note: Introduces repeated field references ({description}
↪  appears three times) and redundant sentence structures
↪  (repetitive "and the {description} indicates that..."
↪  patterns).
SELECT coins.id AS coin_id FROM coins
WHERE category = 'coin' AND status = 'active' AND description is
↪  NOT NULL
AND s'The {description} shows that this cryptocurrency has a big
↪  supply amount and the {description} indicates that this
↪  cryptocurrency is still in circulation and the {description}
↪  can be used to infer that the price of this cryptocurrency is
↪  rising.';
```

Figure 34: Q6 before Expression Reduction

```sql
-- Note: Adds redundant modifiers ("which describes the film
↪  narrative"), repeated field references ({overview} appears
↪  twice), and complex expressions ("after reviewing...for
↪  temporal clues").
SELECT movie_id FROM
    (SELECT s'Based on the movie overview {overview} provided,
    ↪  which describes the film narrative, summarize the
    ↪  geographic and temporal background of the movie,
    ↪  including specific locations and time periods mentioned
    ↪  in this overview {overview}.' AS background, movie_id
        FROM movie
        SEMI JOIN (
            SELECT mg.movie_id
            FROM movie_genres mg JOIN genre g ON mg.genre_id =
            ↪  g.genre_id
            WHERE g.genre_name IN ('Thriller','Comedy','Western')
        ) mg ON m.movie_id = mg.movie_id
        WHERE revenue > budget
        AND length(overview) < 255
    )
    WHERE s'The background summary {background} indicates the
    ↪  story is set in modern times rather than historical
    ↪  periods, and after reviewing the background {background}
    ↪  for temporal clues, this setting classification is
    ↪  confirmed.';
```

Figure 35: Q13 before Expression Reduction